



Comentarios

- Dudas de tarea 1

Inteligencia Artificial

Clase 7: Búsqueda con adversarios



Para el día de hoy

- Juegos
- Minmax
- Búsqueda alfa-beta
- Funciones de evaluación



Al inicio...

- Nuestros agentes estaban solos sus ambientes
- Después vimos que podemos tener una población de individuos que resuelven un problema
- Ahora... encontraremos que en el ambiente ¡hay adversarios!



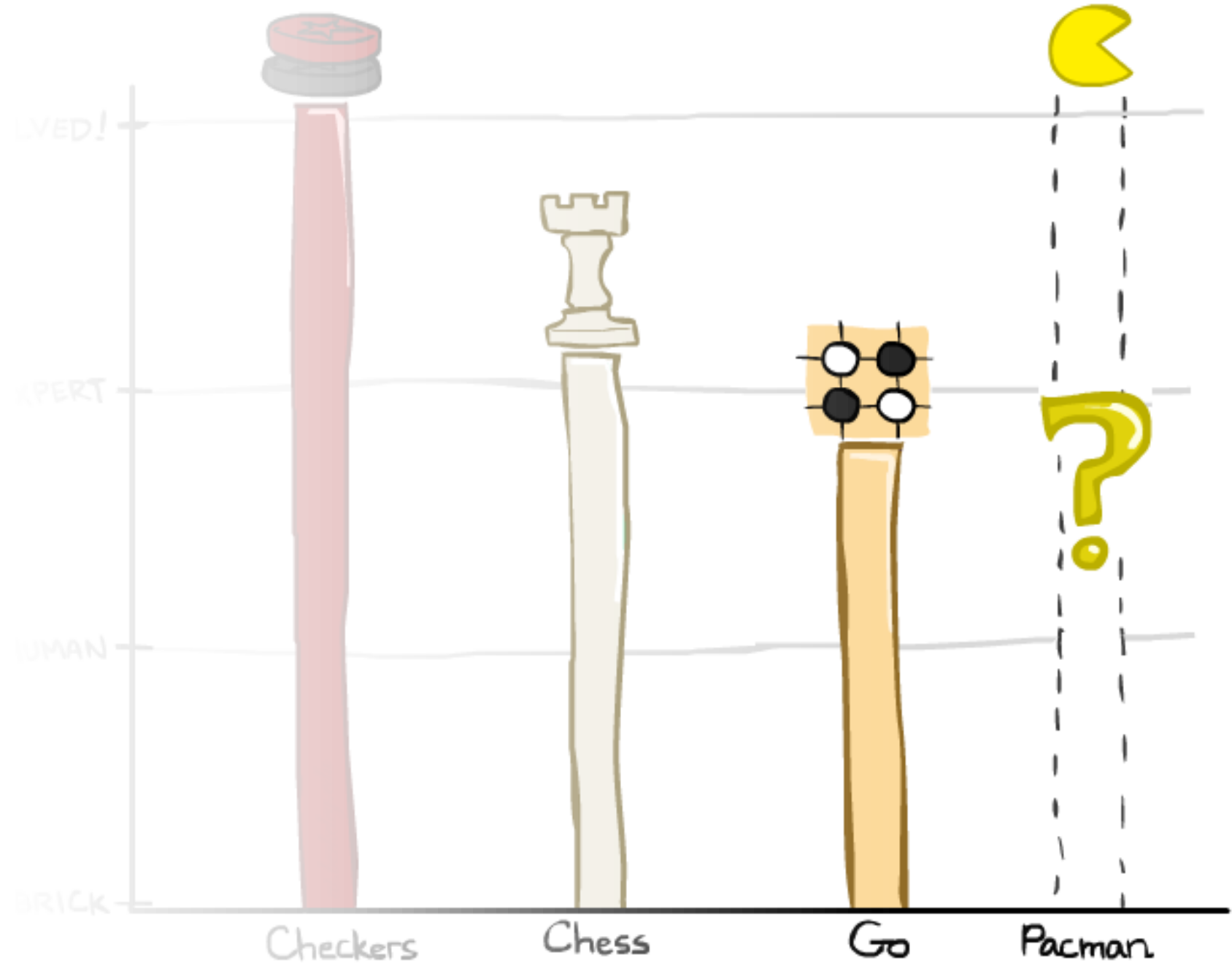
A hand is shown moving a black chess piece on a dark board. Overlaid on the board is a white network diagram consisting of interconnected nodes and lines. A specific path of nodes is highlighted in yellow. The background is a soft, out-of-focus image of the chessboard and network.

Juegos con adversarios

- Existen muchos tipos de juegos
- Ejes
 - Determinista o estocástico
 - Número de jugadores
 - Suma cero
 - Información perfecta
- Queremos algoritmos que calculen la estrategia para movernos en cada estado

El estado del arte en juegos

- Damas inglesas
 - 1950 primera computadora que juega
 - 1994 primer campeón mundial
 - 2007 se resolvió el juego
- Ajedrez
 - 1997 Deep blue derrota al campeón mundial
- Go
 - 2016 Alpha GO derrota un humano campeón
- Pacman
 - ???

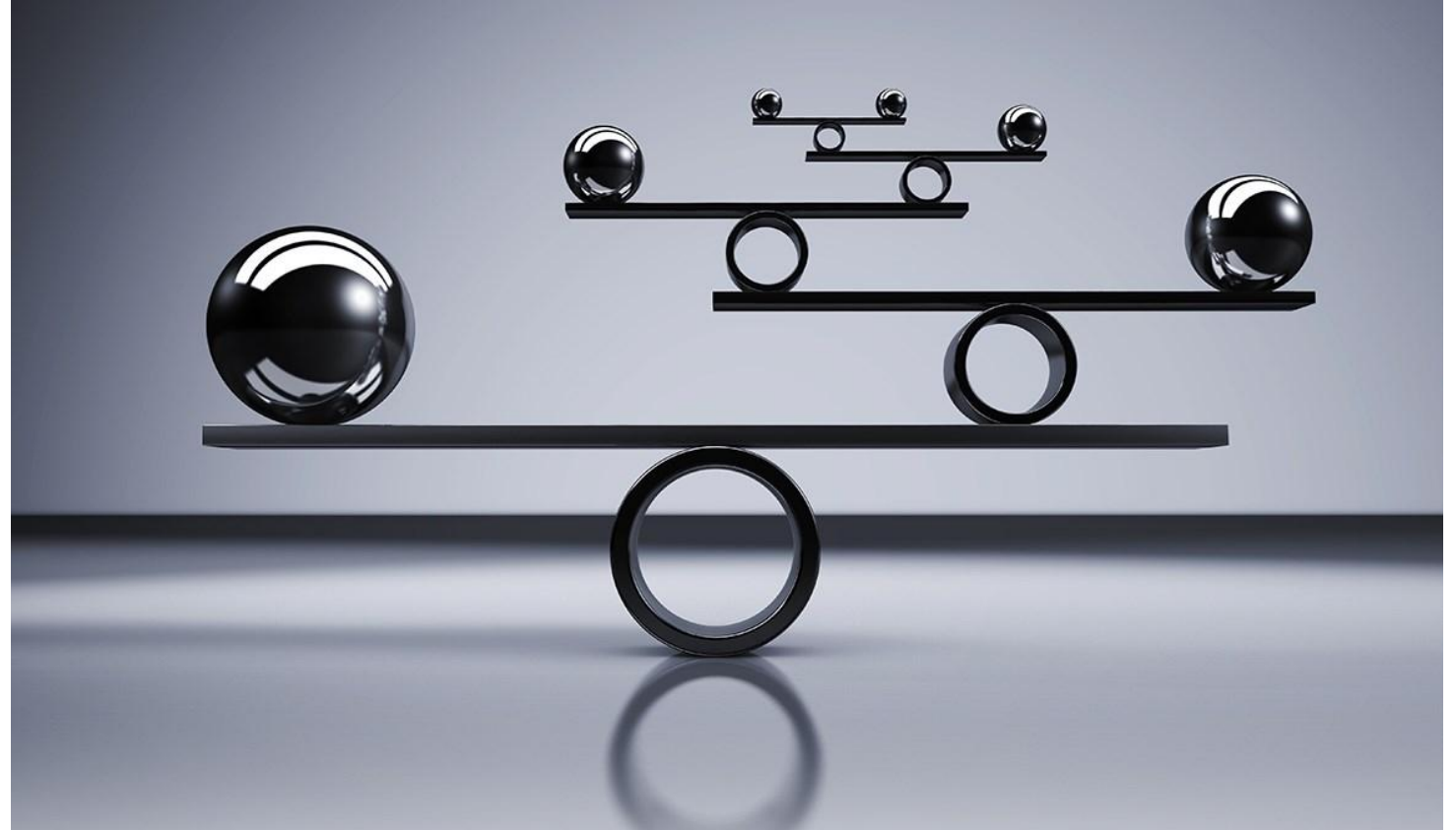


Definición de un juego

- Existen distintas formalizaciones. Un ejemplo
 - Estados: S (inicia en s_0)
 - Jugador: $P = \{1 \dots, N\}$
 - Acciones: A (dependen de cada jugador/estado)
 - Función de transición: $S \times A \rightarrow S$
 - Prueba de meta: $S \rightarrow \{t, f\}$
 - Utilidad terminal: $S \times P \rightarrow R$
- La solución para un jugador es una estrategia $S \rightarrow A$

Juegos

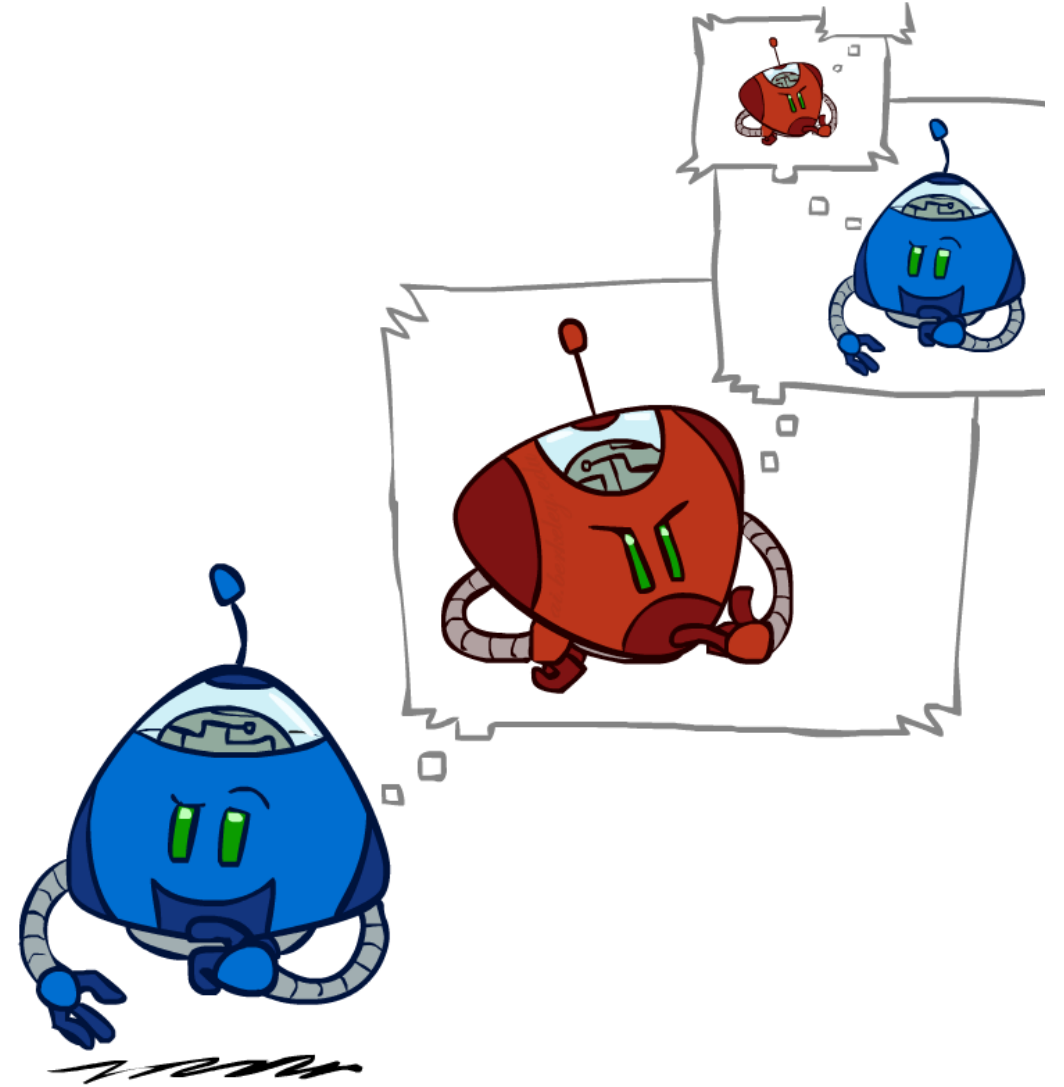
- Juegos de suma cero
 - Agentes que tienen utilidades opuestas
 - Alguien gana y alguien pierde en la misma cantidad
 - Es pura competencia
- Juegos en general
 - Los agentes tienen utilidades independientes
 - Existe cooperación, indiferencia, competencia y otras posibilidades

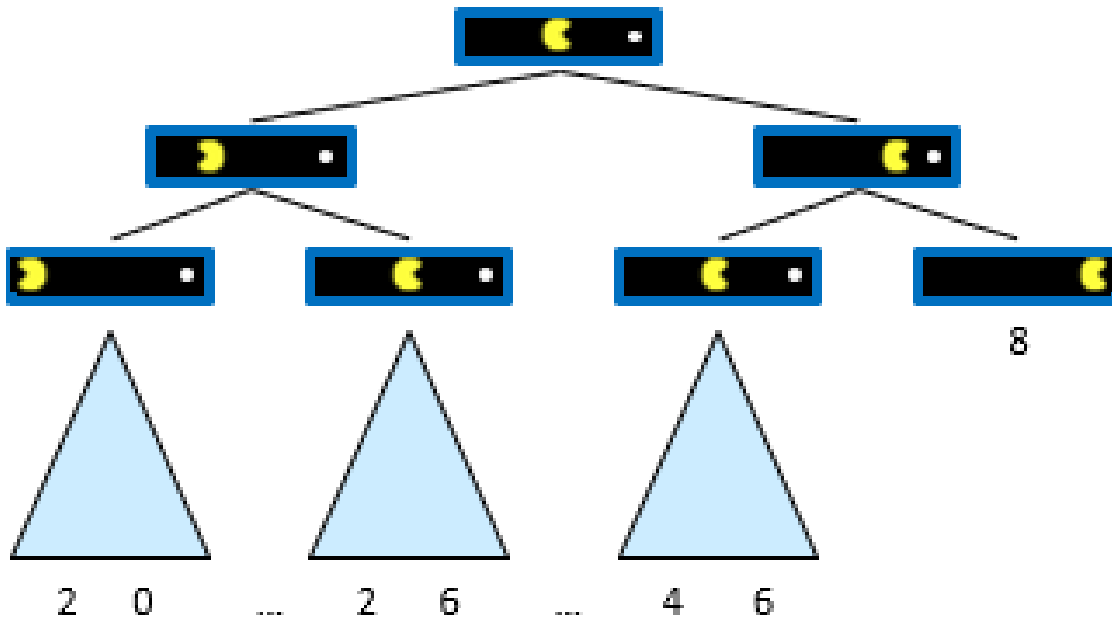


Búsqueda con adversarios

No solo es decidir que haré de forma independiente para decidir debo pensar mis opciones, las de mi oponente y pensar...

- ¿Qué hará mi oponente?
- ¿Qué pensará mi oponente que haré yo ?
- ¿Qué pensará mi oponente que pensaré yo que hará él ?
- :





El valor de un estado

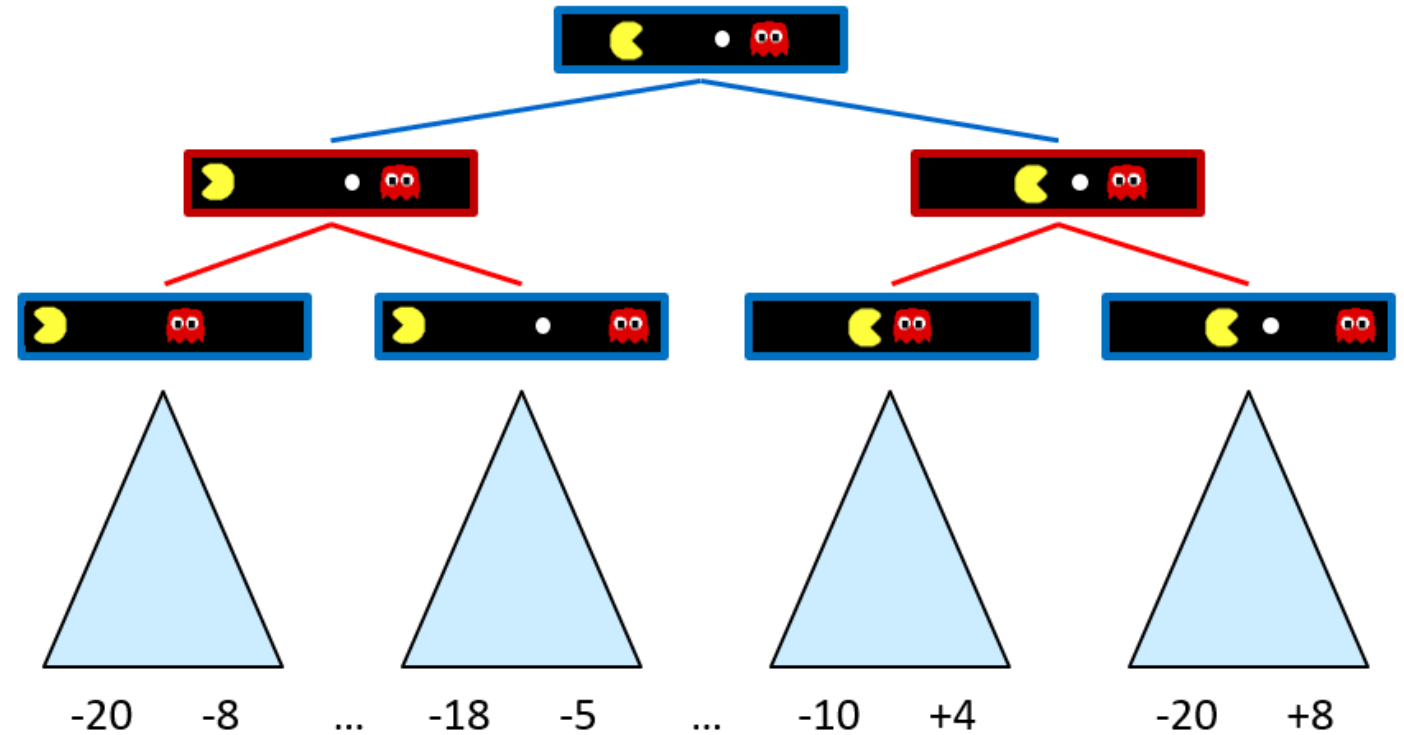
Es el mejor resultado (utilidad) que se puede obtener desde ese estado

- En estados terminales: $V(s) = x$, donde x es un valor conocido
- En estados no terminales:

$$V(s) = \max_{s' \in ch(s)} V(s')$$

Árboles de juegos con adversarios

- No se tiene control del árbol completo
- Solo se puede seleccionar las decisiones propias
- Podemos suponer que lo peor va a suceder



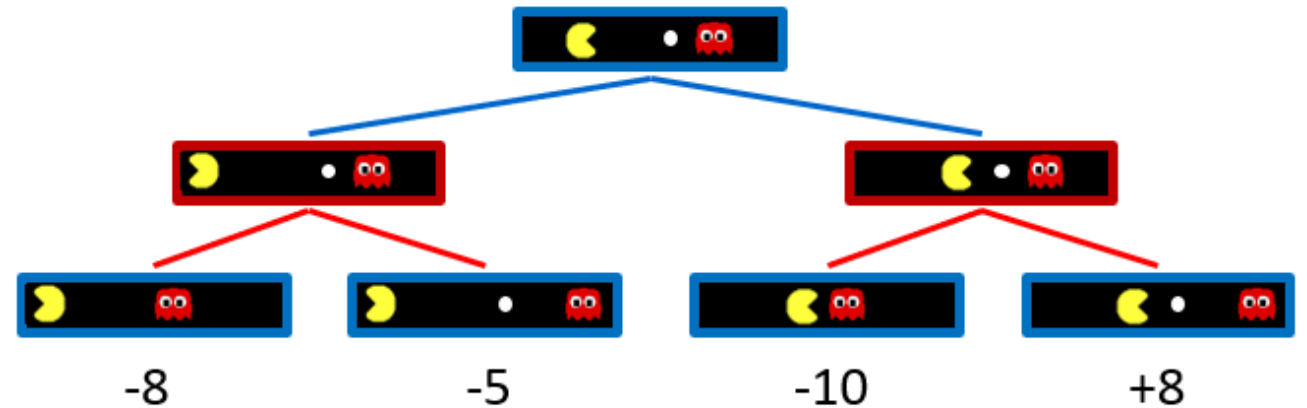
Valores minimax

- Estados terminales
 - $V(s) = x$ donde x es conocido
- Estados bajo el control del oponente

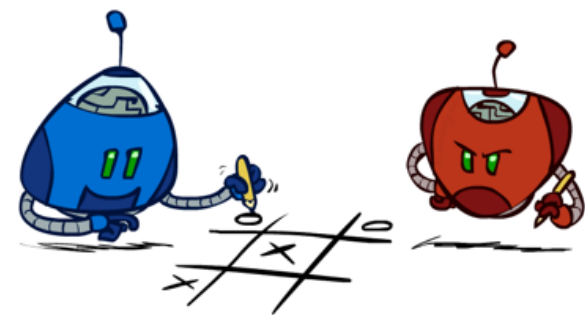
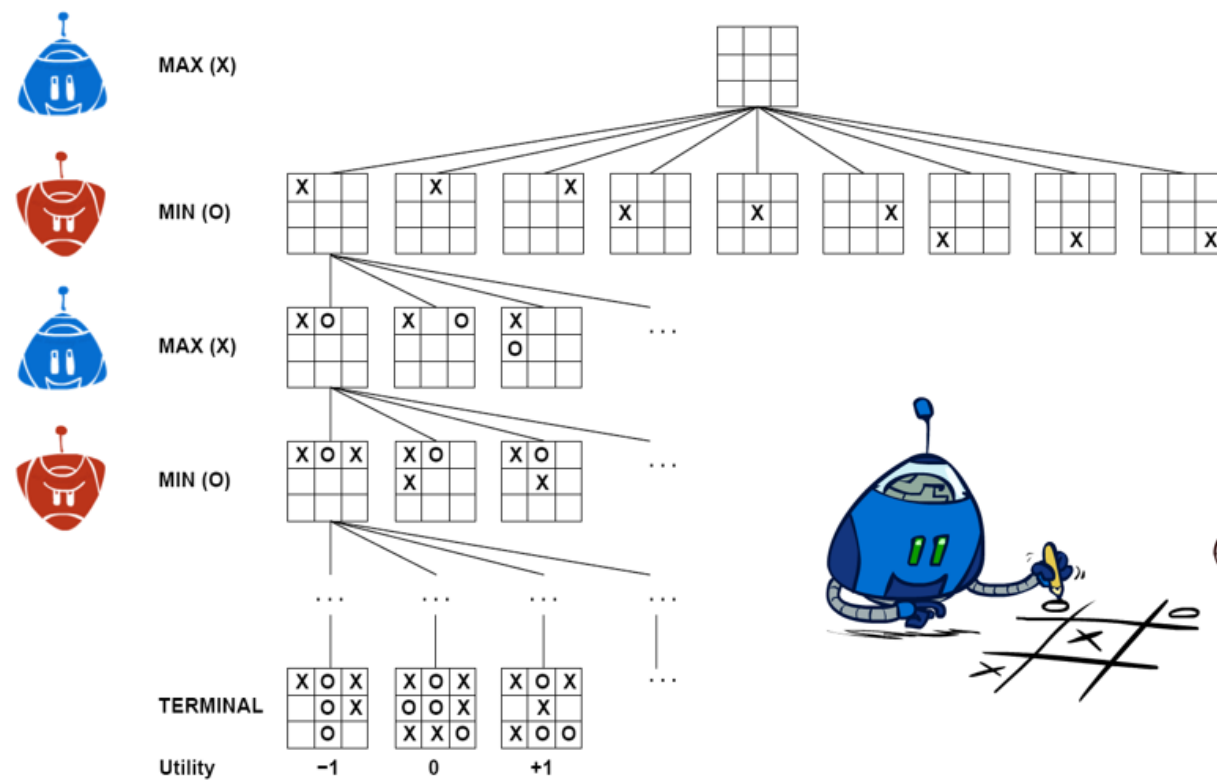
$$V(s) = \min_{s' \in \text{suc}(s')} V(s')$$

- Estados bajo el control del agente

$$V(s) = \max_{s' \in \text{suc}(s)} V(s')$$

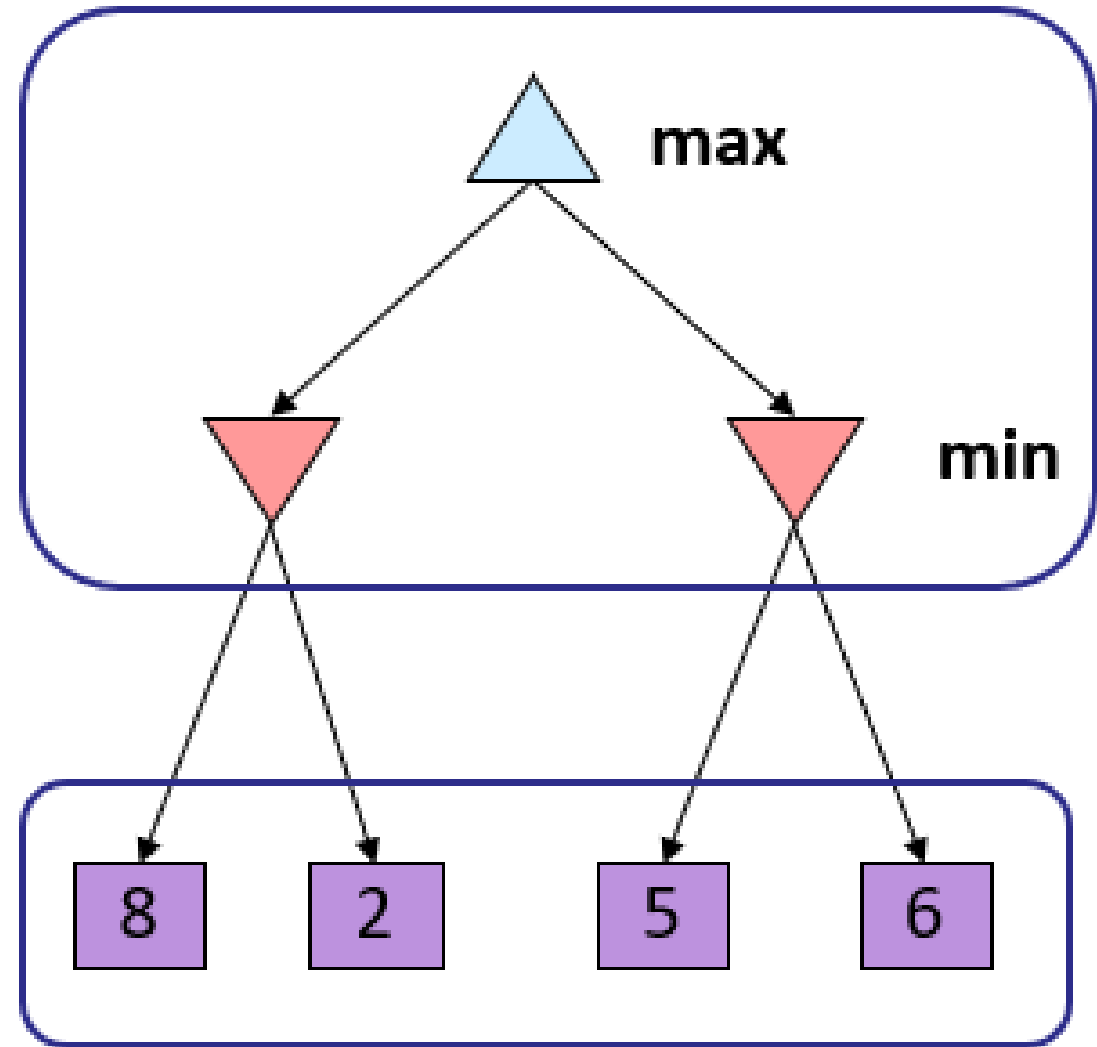


Un árbol para gato



Búsqueda con adversarios (minimax)

- Juegos deterministas con suma cero
 - Gato, ajedrez, damas inglesas
 - Un jugador maximiza su resultado
 - El otro jugador minimiza el resultado
- Minimax
 - El espacio de estados es un árbol
 - Cada jugador tiene un turno
 - Calcular el valor minimax para cada nodo



Implementación de minimax

def max-value(state):

initialize $v = -\infty$

for each successor of state:

$v = \max(v, \text{min-value}(\text{successor}))$

return v

$$V(s) = \max_{s' \in \text{successors}(s)} V(s')$$



def min-value(state):

initialize $v = +\infty$

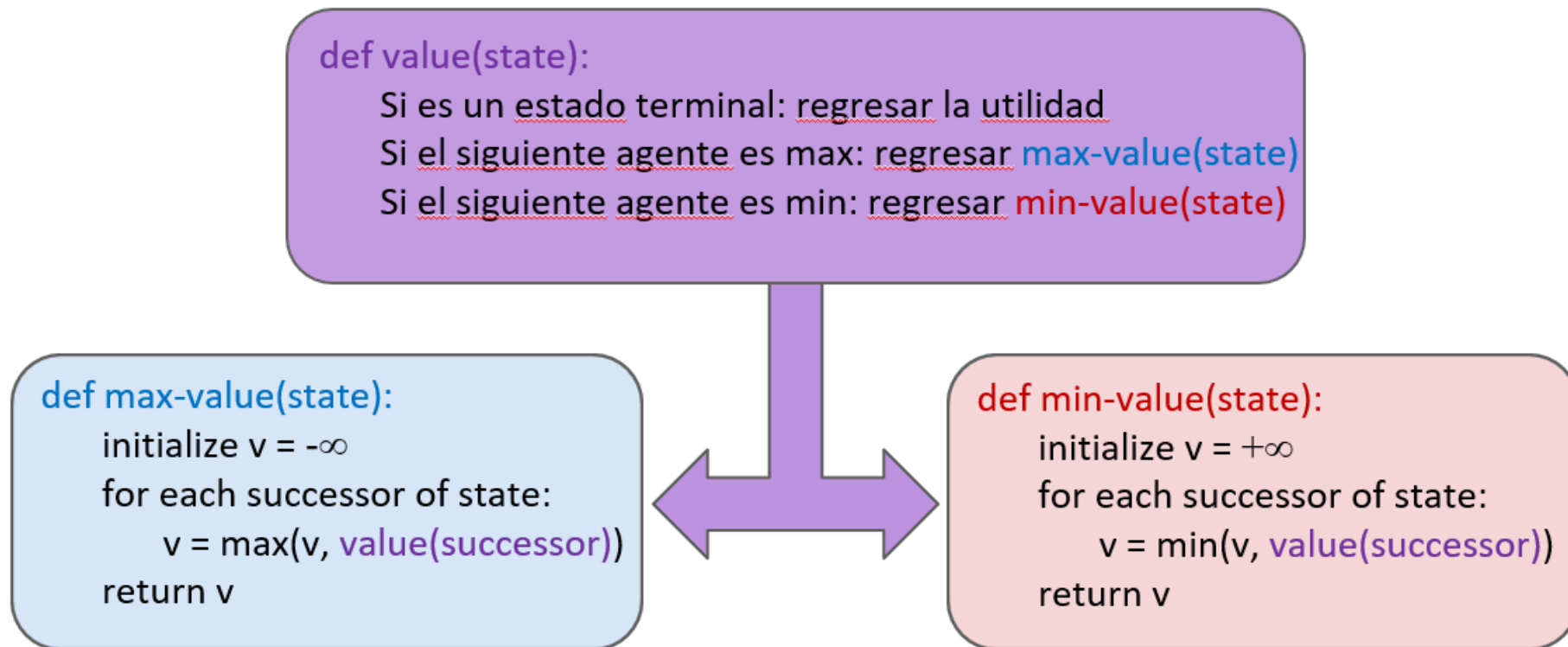
for each successor of state:

$v = \min(v, \text{max-value}(\text{successor}))$

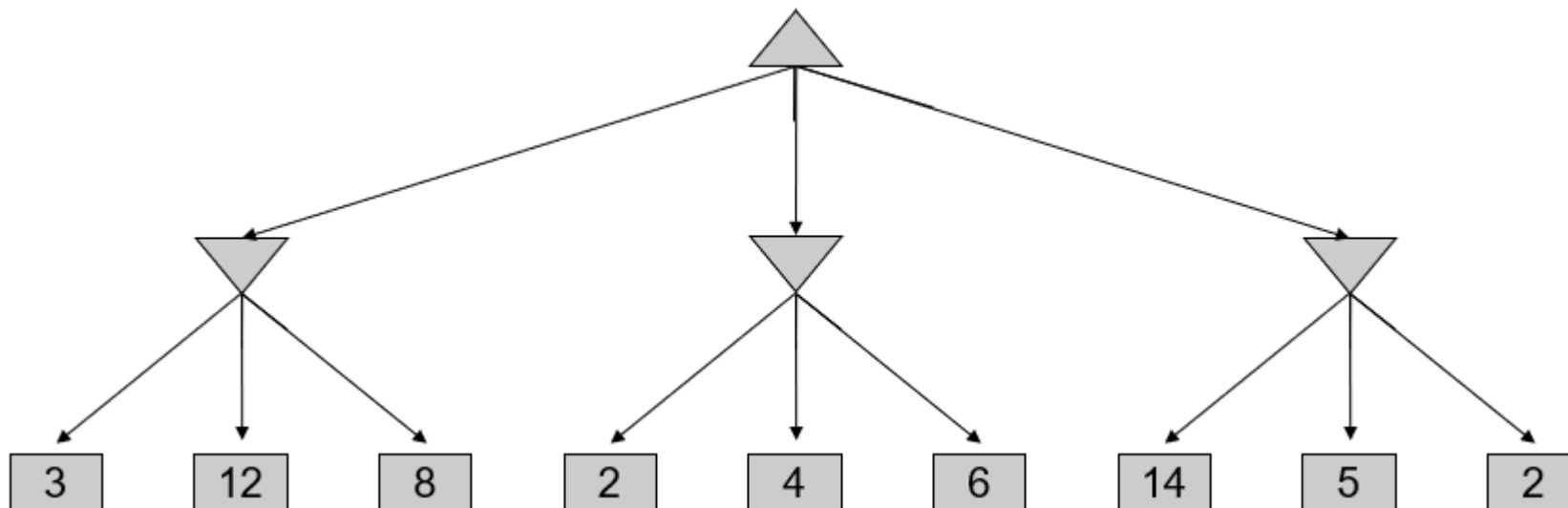
return v

$$V(s') = \min_{s \in \text{successors}(s')} V(s)$$

Minimax implementación con un desapachador



Ejemplo minimax



Propiedades de minimax



¿Completo?



¿Optimo?



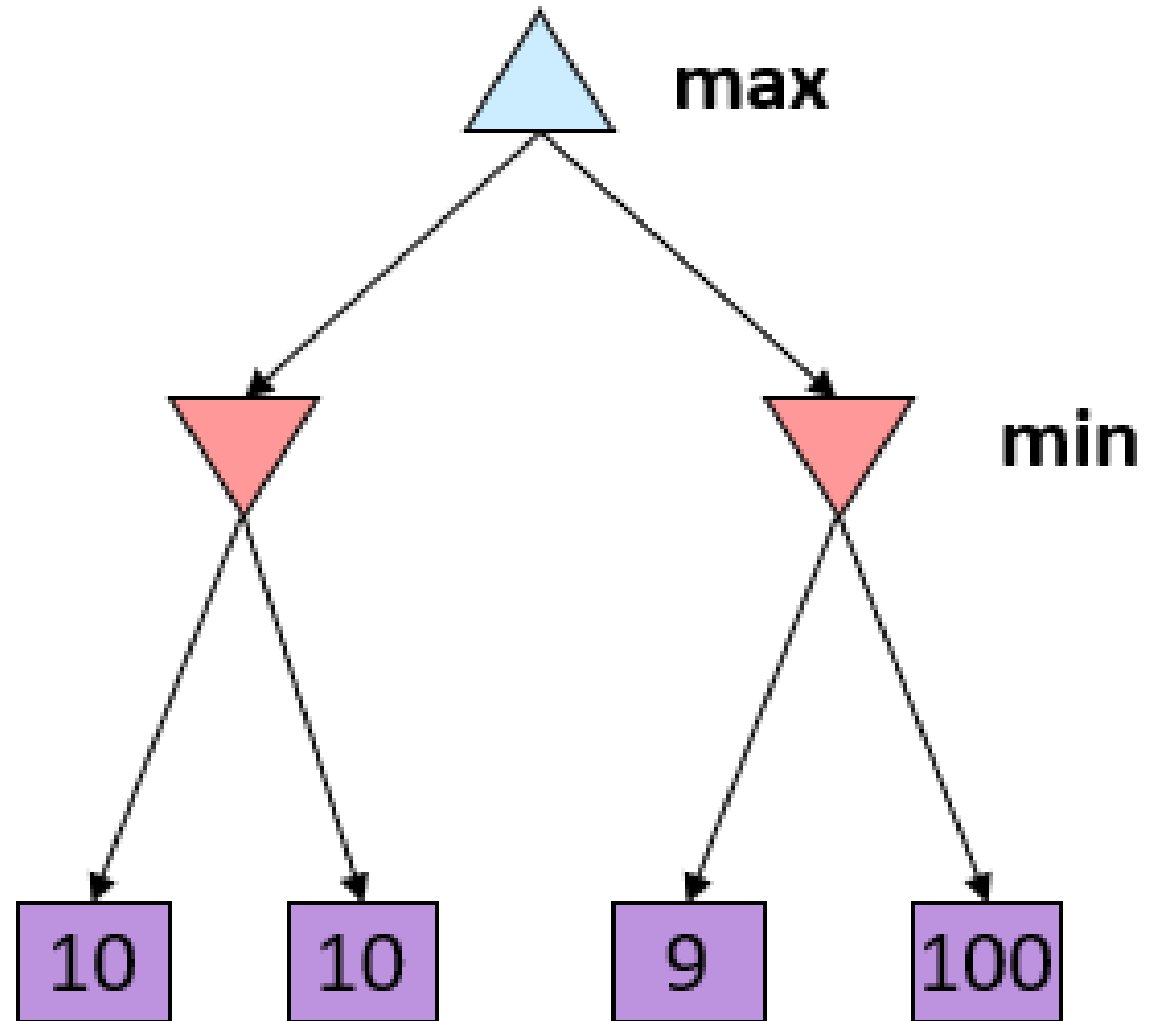
¿Complejidad de tiempo?



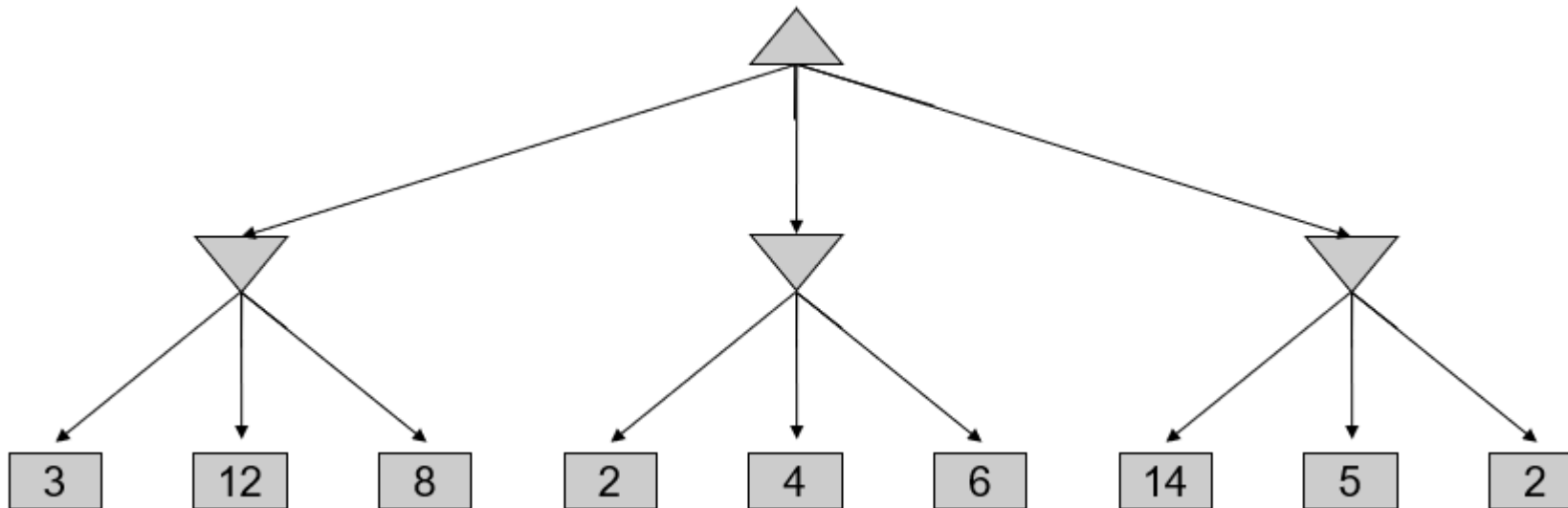
¿Complejidad de espacio?

Optimalidad

- ¿Cuál camino eligen?

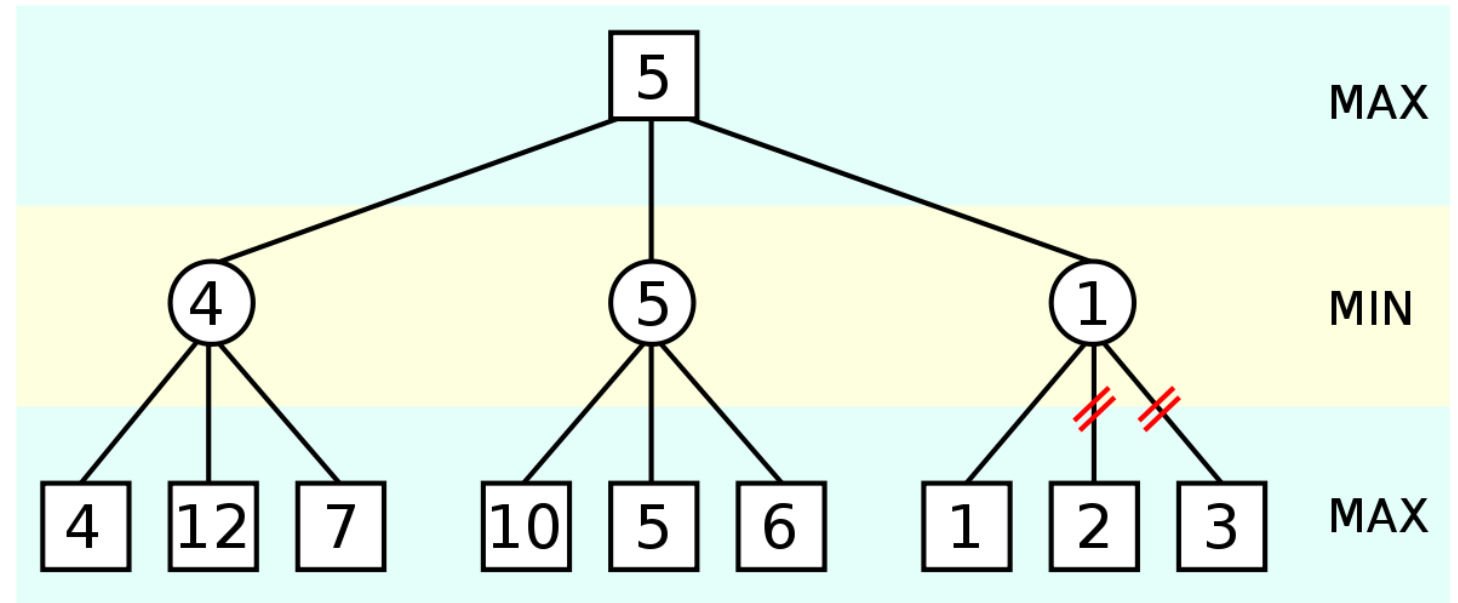


Regresemos a nuestro ejemplo



Algoritmo de podado alfa-beta

- Se calcula el valor mínimo para algún nodo n
- Se cicla para los hijos de n
- El estimado del min de los hijos de n baja
- Sea α el mejor valor que max puede obtener en el recorrido actual
- Si n es peor que α max no lo va a considerar y podemos no explorar esa región



Implementación del algoritmo

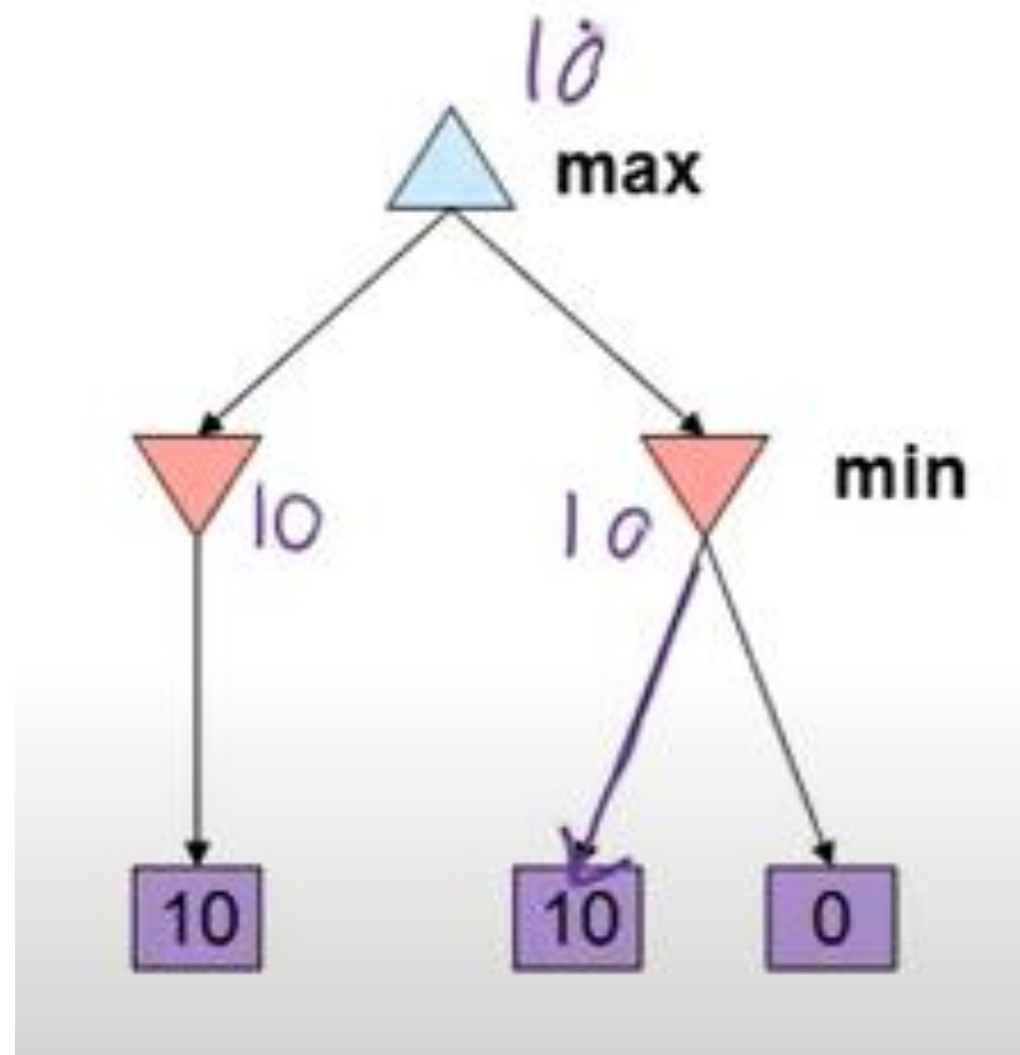
α : mejor camino a la raíz de max
 β : major camino a la raíz de min

```
def max-value(state,  $\alpha$ ,  $\beta$ ):  
    initialize  $v = -\infty$   
    for each successor of state:  
         $v = \max(v, \text{value}(\text{successor}, \alpha, \beta))$   
        if  $v \geq \beta$  return  $v$   
         $\alpha = \max(\alpha, v)$   
    return  $v$ 
```

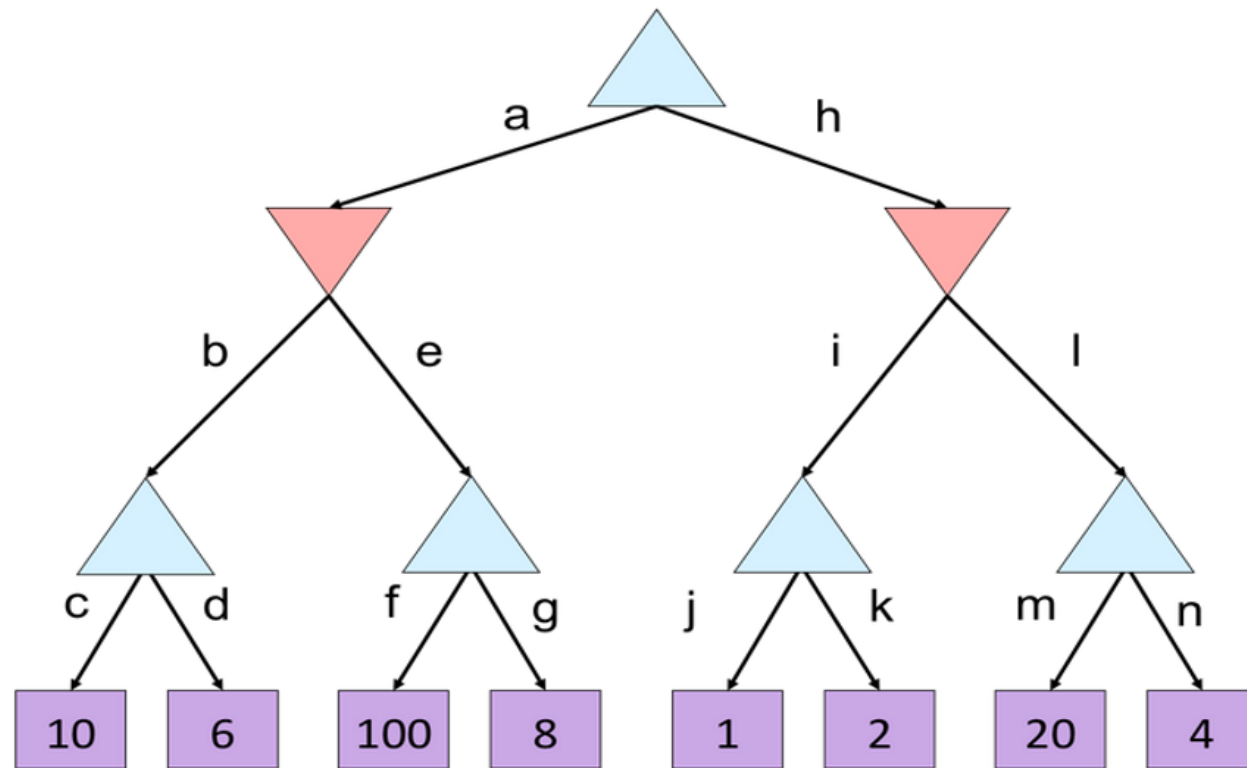
```
def min-value(state,  $\alpha$ ,  $\beta$ ):  
    initialize  $v = +\infty$   
    for each successor of state:  
         $v = \min(v, \text{value}(\text{successor}, \alpha, \beta))$   
        if  $v \leq \alpha$  return  $v$   
         $\beta = \min(\beta, v)$   
    return  $v$ 
```

Propiedades

- El podado no tiene efecto en el valor minimax de la raíz
- Los valores de nodos intermedio pueden tener errores
- Complejidad de tiempo $O(b^{\frac{m}{2}})$
- El orden importa



Un ejercicio



Limitantes en recursos

- En juegos realistas, no se puede llegar a las hojas
- ¿Ideas?

Limitantes en recursos

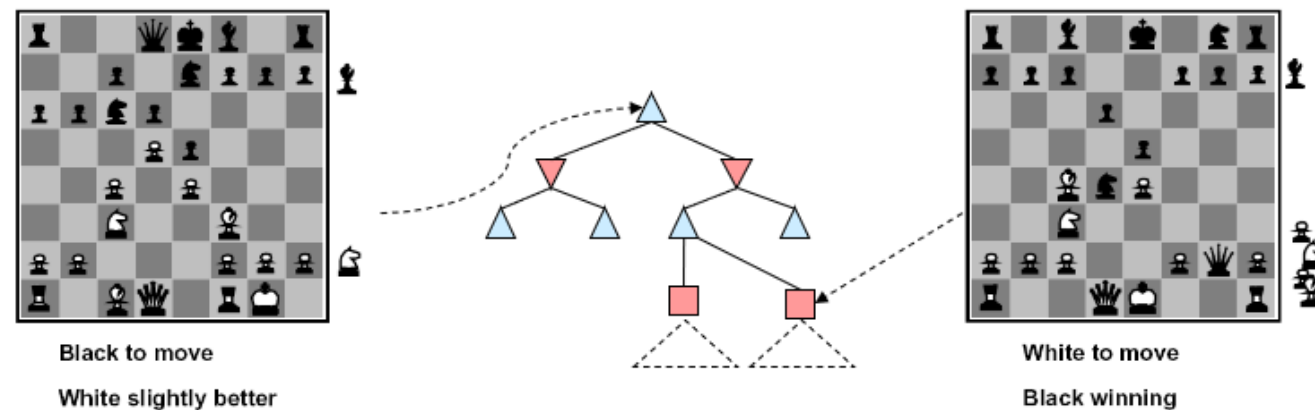
- En juegos realistas, no se puede llegar a las hojas
- ¿Ideas?
 - Buscar solo a una profundidad definida
 - Reemplazar utilidades terminales con una evaluación en nodos no terminales (similar al rol de una heurística en A*)
 - Utilizar búsqueda iterativa para un algoritmo siempre listo
- Ejemplo
 - Hacer búsqueda en ajedrez viendo "8 movimientos al futuro"

La profundidad de la búsqueda tiene impacto

- La evaluación siempre es imperfecta
- Se necesita
- A mayor profundidad de búsqueda menos relevante es la calidad de la evaluación de la función pero mucho más caro

Evaluación de funciones

- Función idea: regresar el valor de minimax de la posición actual
- Nos conformamos con: una noción que nos diga el valor de nuestra posición. En general una función $F: \mathbb{R}^n \rightarrow \mathbb{R}^k$, donde n son las dimensiones a considerar y k son los aspectos que estamos considerando.
- Dado que F es una función vectorial, se puede elegir:
 - $\tilde{F}(s) = w_1 f_1(s) + w_2 f_2(s) + \dots + w_n f_n(s)$
- Donde las $w_i \in [0,1]$ representan la relevancia y $f_i: \mathbb{R}^N \rightarrow \mathbb{R}$ para $i = 1, \dots, k$



Para la otra vez...

- Incertidumbre



iimas

The End.